JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.reflect

Class Parameter

java.lang.Object
    java.lang.reflect.Parameter

All Implemented Interfaces:

AnnotatedElement

```
public final class Parameter
extends Object
implements AnnotatedElement
```

Information about method parameters. A `Parameter` provides information about method parameters, including its name and modifiers. It also provides an alternate means of obtaining attributes for the parameter.

Since:

1.8

Method Summary

Concrete Methods

Modifier and Type	Method	De
boolean	**equals**(**Object** obj)	C b e th
AnnotatedType	**getAnnotatedType**()	R A ol

		re us to ty fo pa re th Pa
<T extends **Annotation**> T	**getAnnotation**(**Class**<T> annotationClass)	R el ar th ty ar *p* n
Annotation[]	**getAnnotations**()	R ar th *p* el
<T extends **Annotation**> T[]	**getAnnotationsByType**(**Class**<T> annotationClass)	R ar th *as* w el
<T extends **Annotation**> T	**getDeclaredAnnotation**(**Class**<T> annotationClass)	R el ar th ty ar *d* *p* n
Annotation[]	**getDeclaredAnnotations**()	R ar th *d* *p* el
	getDeclaredAnnotationsByType(**Class**<T> annotationClass)	

<T extends **Annotation**> T[]		R el ar fo sp if ar ar *d p in p*
Executable	**getDeclaringExecutable**()	R E w th p
int	**getModifiers**()	G m fo p re th o
String	**getName**()	R n p
Type	**getParameterizedType**()	R o id p ty p re th o
Class<?>	**getType**()	R C th th ty p re

		th ol
int	**hashCode**()	R co th ex ha th
boolean	**isImplicit**()	R if pa in de so re ot
boolean	**isNamePresent**()	R th ha ac th re ot
boolean	**isSynthetic**()	R if pa no in ex de so re ot
boolean	**isVarArgs**()	R if pa re va ar re ot
String	**toString**()	R st

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.lang.reflect.AnnotatedElement

isAnnotationPresent

Method Detail

equals

```
public boolean equals(Object obj)
```

Compares based on the executable and the index.

Overrides:

equals in class Object

Parameters:

obj - The object to compare.

Returns:

Whether or not this is equal to the argument.

See Also:

Object.hashCode(), HashMap

hashCode

```
public int hashCode()
```

Returns a hash code based on the executable's hash code and the index.

Overrides:

`hashCode` in class `Object`

Returns:

A hash code based on the executable's hash code.

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

isNamePresent

```
public boolean isNamePresent()
```

Returns true if the parameter has a name according to the class file; returns false otherwise. Whether a parameter has a name is determined by the MethodParameters attribute of the method which declares the parameter.

Returns:

true if and only if the parameter has a name according to the class file.

toString

```
public String toString()
```

Returns a string describing this parameter. The format is the modifiers for the parameter, if any, in canonical order as recommended by *The Java™ Language Specification*, followed by the fully- qualified type of the parameter (excluding the last [] if the parameter is variable arity), followed by "..." if the parameter is variable arity, followed by a space, followed by the name of the parameter.

Overrides:

`toString` in class `Object`

Returns:

A string representation of the parameter and associated information.

getDeclaringExecutable

```
public Executable getDeclaringExecutable()
```

Return the `Executable` which declares this parameter.

Returns:

`The Executable declaring this parameter.`

getModifiers

```
public int getModifiers()
```

Get the modifier flags for this the parameter represented by this `Parameter` object.

Returns:

`The modifier flags for this parameter.`

getName

```
public String getName()
```

Returns the name of the parameter. If the parameter's name is present, then this method returns the name provided by the class file. Otherwise, this method synthesizes a name of the form argN, where N is the index of the parameter in the descriptor of the method which declares the parameter.

Returns:

`The name of the parameter, either provided by the class file or synthesized if the class file does not provide a name.`

getParameterizedType

```
public Type getParameterizedType()
```

Returns a `Type` object that identifies the parameterized type for the parameter represented by this `Parameter` object.

Returns:

`a Type object identifying the parameterized type of the parameter represented by this object`

getType

```
public Class<?> getType()
```

Returns a `Class` object that identifies the declared type for the parameter represented by this `Parameter` object.

Returns:

a Class object identifying the declared type of the parameter represented by this object

getAnnotatedType

```
public AnnotatedType getAnnotatedType()
```

Returns an AnnotatedType object that represents the use of a type to specify the type of the formal parameter represented by this Parameter.

Returns:

an AnnotatedType object representing the use of a type to specify the type of the formal parameter represented by this Parameter

isImplicit

```
public boolean isImplicit()
```

Returns `true` if this parameter is implicitly declared in source code; returns `false` otherwise.

Returns:

true if and only if this parameter is implicitly declared as defined by *The Java™ Language Specification*.

isSynthetic

```
public boolean isSynthetic()
```

Returns `true` if this parameter is neither implicitly nor explicitly declared in source code; returns `false` otherwise.

Returns:

true if and only if this parameter is a synthetic construct as defined by *The Java™ Language Specification*.

See *The Java™ Language Specification*:

13.1 The Form of a Binary

isVarArgs

```
public boolean isVarArgs()
```

Returns `true` if this parameter represents a variable argument list; returns `false` otherwise.

Returns:

`true if an only if this parameter represents a variable argument list.`

getAnnotation

```
public <T extends Annotation> T getAnnotation(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Specified by:

`getAnnotation` in interface `AnnotatedElement`

Type Parameters:

`T - the type of the annotation to query for and return if present`

Parameters:

`annotationClass - the Class object corresponding to the annotation type`

Returns:

`this element's annotation for the specified annotation type if present on this element, else null`

Throws:

`NullPointerException - if the given annotation class is null`

getAnnotationsByType

```
public <T extends Annotation> T[] getAnnotationsByType(Class<T> annotationClass)
```

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

getAnnotationsByType in interface AnnotatedElement

Type Parameters:

T - the type of the annotation to query for and return if present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if associated with this element, else an array of length zero

Throws:

NullPointerException - if the given annotation class is null

getDeclaredAnnotations

```
public Annotation[] getDeclaredAnnotations()
```

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

getDeclaredAnnotations in interface AnnotatedElement

Returns:

annotations directly present on this element

getDeclaredAnnotation

```
public <T extends Annotation> T getDeclaredAnnotation(Class<T> annotationClass)
```

Description copied from interface: AnnotatedElement

Returns this element's annotation for the specified type if such an annotation is *directly present,* else null. This method ignores inherited annotations. (Returns null if no annotations are directly present on this element.)

Specified by:

getDeclaredAnnotation in interface AnnotatedElement

Type Parameters:

T - the type of the annotation to query for and return if directly present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if directly present on this element, else null

Throws:

NullPointerException - if the given annotation class is null

getDeclaredAnnotationsByType

public <T extends Annotation> T[] getDeclaredAnnotationsByType(Class<T> annotatio

Description copied from interface: AnnotatedElement

Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*. This method ignores inherited annotations. If there are no specified annotations directly or indirectly present on this element, the return value is an array of length 0. The difference between this method and AnnotatedElement.getDeclaredAnnotation(Class) is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation if one is present. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

getDeclaredAnnotationsByType in interface AnnotatedElement

Type Parameters:

T - the type of the annotation to query for and return if directly or indirectly present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if directly or indirectly present on this element, else an array of length zero

Throws:

NullPointerException - if the given annotation class is null

getAnnotations

```
public Annotation[] getAnnotations()
```

Returns annotations that are *present* on this element. If there are no annotations *present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getAnnotations` in interface `AnnotatedElement`

Returns:

annotations present on this element

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.